

P3468R0

2024-10 Library Evolution Poll Outcomes

Published Proposal, 2024-12-16

Authors:

[Inbal Levi - Library Evolution Chair](#) (MPGC Services LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Ben Craig - Library Evolution Assistant Chair](#) (Raven)

[Billy Baker - Library Evolution Incubator Chair](#) (NVIDIA)

[Nevin Liber - Library Evolution Incubator Assistant Chair and Admin Chair](#) (Argonne National Laboratory)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

- 3.1 Poll 1: Send "[P2897R5] aligned_accessor: An mdspan accessor expressing pointer over-alignment" to Library Working Group for C++26.
- 3.2 Poll 2: Send "[P3008R2] Atomic floating-point min/max" to Library Working Group for C++26.
- 3.3 Poll 3: Send "[P3050R2] Fix C++26 by optimizing linalg::conjugated for noncomplex value types" to Library Working Group for C++26.

- 3.4 Poll 4: Send "[P3068R4] Allowing exception throwing in constant-evaluation" to Library Working Group for C++26.
- 3.5 Poll 5: Send "[P3136R0] Retiring niebloids" to Library Working Group for C++26.
- 3.6 Poll 6: Send "[P3222R0] Fix C++26 by adding transposed special cases for P2642 layouts" to Library Working Group for C++26.
- 3.7 Poll 7: Send "[P3309R2] constexpr atomic and atomic_ref" to Library Working Group for C++26.
- 3.8 Poll 8: Send "[P3323R0] cv-qualified types in atomic and atomic_ref" to Library Working Group for C++26.
- 3.9 Poll 9: Send "[P3325R3] A Utility for Creating Execution Environments" to Library Working Group for C++26.
- 3.10 Poll 10: Send "[P0472R2] Put std::monostate in " to Library Working Group for C++26.
- 3.11 Poll 11: Send "[P3355R1] Fix submdspan for C++26" to Library Working Group for C++26.
- 3.12 Poll 12: Send "[P3369R0] constexpr for uninitialized_default_construct" to Library Working Group for C++26.
- 3.13 Poll 13: Approve "[P3399R0] Adjusting Electronic Polls to a Hybrid Workflow" and apply the change to the process of Library Evolution Working Group.

References

Informative References

§ 1. Introduction

In 2024-10, the C++ Library Evolution group conducted a series of electronic decision polls [\[P3467R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 21 people participated in the polls. Some participants opted to not vote on some polls (lower participation was at: P2897R5, P3050R2, P3222R0, P3355R1) Thank you to everyone who participated, and to the papers' authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send "[P2897R5] aligned_accessor: An mdspan accessor expressing pointer overalignment" to Library Working Group for C++26.	9	3	0	0	0	Strong consensus in favor
Poll 2: Send "[P3008R2] Atomic floating-point min/max" to Library Working Group for C++26.	7	4	3	0	1	Consensus in favor
Poll 3: Send "[P3050R2] Fix C++26 by optimizing linalg::conjugated for noncomplex value types" to Library Working Group for C++26.	9	2	0	0	0	Strong consensus in favor
Poll 4: Send "[P3068R4] Allowing exception throwing in constant-evaluation" to Library Working Group for C++26.	13	3	0	1	0	Consensus in favor
Poll 5: Send "[P3136R0] Retiring niebloids" to Library Working Group for C++26.	10	4	0	1	1	Consensus in favor
Poll 6: Send "[P3222R0] Fix C++26 by adding transposed special cases for P2642 layouts" to Library Working Group for C++26.	7	3	0	0	0	Strong consensus in favor
Poll 7: Send "[P3309R2] constexpr atomic and atomic_ref" to Library Working Group for C++26.	11	6	1	1	0	Consensus in favor
Poll 8: Send "[P3323R0] cv-qualified types in atomic and atomic_ref" to Library Working Group for C++26.	9	8	1	1	0	Consensus in favor
Poll 9: Send "[P3325R3] A Utility for Creating Execution Environments" to Library Working Group for C++26.	7	5	3	0	0	Strong consensus in favor
Poll 10: Send "[P0472R2] Put std::monostate in " to Library Working Group for C++26.	10	5	4	0	0	Strong consensus in favor
Poll 11: Send "[P3355R1] Fix submdspan for C++26" to Library Working Group for C++26.	8	3	0	0	0	Strong consensus in favor
Poll 12: Send "[P3369R0] constexpr for uninitialized_default_construct" to Library Working Group for C++26.	16	3	0	0	0	Strong consensus in favor
Poll 13: Approve "[P3399R0] Adjusting Electronic Polls to a Hybrid Workflow" and apply the change	11	4	1	2	1	Consensus in favor

to the process of Library Evolution Working Group.

All the polls have consensus in favor and will be forwarded to LWG for C++26, accept from P3399R0 which will be applied to modify LEWG procedure.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[P2897R5] aligned_accessor: An mdspan accessor expressing pointer overalignment" to Library Working Group for C++26.

The paper is very well-written and explains the rationale and technical bits with very high quality. It's another good example of well-done homework.

— Strongly Favor

This feature lets mdspan users express that the beginning of an array (not necessarily even a multi-dimensional array -- it works for rank-1 arrays too) is overaligned. This lets users express interface and hardware requirements, and also exposes optimization opportunities to compilers. The design discussion at St. Louis helped clarify mdspan's opportunities to improve safety in realistic ways.

— Strongly Favor

Much of mdspan's utility is tied up in being easily vectorizable, so this paper makes sense to go to C++26.

— Weakly Favor

§ 3.2. Poll 2: Send "[P3008R2] Atomic floating-point min/max" to Library Working Group for C++26.

This exposes a feature increasingly accelerated in hardware, while at the same time clarifying the behavior of min and max with respect to floating-point numbers.

— Strongly Favor

Small but useful utility, that is hard to provide outside of the standard.

— Weakly Favor

Seems like the technical problems of earlier approaches have been solved. I can't find any mention of implementation experience in the paper, so N instead of WF.

— Neutral

The provided semantic still does not guarantee that two threads calling `fetch_min` with arguments `-0` and `+0` in a loop, will not produce a spin loop, by replacing each other results. I.e. due unspecified ordering, we may end up in situation where: Thread 1: writes `-0` Thread 2: writes `+0` Thread 1: writes `-0` Thread 2: writes `+0`

— Strongly Against

§ 3.3. Poll 3: Send "[P3050R2] Fix C++26 by optimizing `linalg::conjugated` for noncomplex value types" to Library Working Group for C++26.

This makes providing specialized algorithms for certain use cases easier and reduces the compilation time and template bloat.

— Strongly Favor

This simplifies both implementations of `std::linalg` (P1673R13, approved for C++26) and users' libraries that use components of `std::linalg`. It reduces the number of template instantiations needed to make common linear algebra code patterns work correctly and efficiently.

— Strongly Favor

§ 3.4. Poll 4: Send "[P3068R4] Allowing exception throwing in constant-evaluation" to Library Working Group for C++26.

This is a significant step forward for constant evaluation in C++ and I expect to see the use of this functionality in papers in the very near future.

— Strongly Favor

This will make it possible to write code using static reflection (P2996 and follow-on proposals) in a way that looks as much like existing C++ code as possible. I particularly appreciate the paper's examples, that make its effects on new code and impact on existing code clear.

— Strongly Favor

The "partial implementation" mentioned in the paper is something that worries me. However, I believe the errors can be still caught (if any) given that we have time for C++26 timeframe.

— Weakly Favor

I am concerned that using exceptions during constant evaluation will make code less maintainable, not more. We need usage experience with reflection before making language change of that nature.

— Weakly Against

§ 3.5. Poll 5: Send "[P3136R0] Retiring niebloids" to Library Working Group for C++26.

This makes using these CPO as functions much more convenient and standard-compliant. As this reflects existing practice, such uses were already portable.

— Strongly Favor

We can call these things function objects now. There's no reason not to. In general, this helps make the meaning of the Standard more clear and its implementation strategy less ambiguous.

— Strongly Favor

Hopefully, this will align the spec with what exists, and if we would like to modify by adding a language feature we can re-define/re-add the term.

— Weakly Favor

I would still rather not close out the possibility of adding core language features for implementing niebloids in future.

— Weakly Against

We should wait for a language solution.

— Strongly Against

§ 3.6. Poll 6: Send "[P3222R0] Fix C++26 by adding transposed special cases for P2642 layouts" to Library Working Group for C++26.

This has the same motivation as P3050, just for the "transposed" function (that affects layouts) instead of the "conjugated" function (that affects accessors). Approving P32222 will simplify both implementations of `std::linalg` and users' libraries that use "transposed."

— Strongly Favor

Useful for my work.

— Strongly Favor

I was not part of the discussion, but quickly looking into the paper, not adding that now will lead to a breaking change. I trust authors here.

— Weakly Favor

§ 3.7. Poll 7: Send "[P3309R2] constexpr atomic and atomic_ref" to Library Working Group for C++26.

Seems straightforward given we do not have multithreading during constant evaluation. This will unlock constexpr shared_ptr and many other data structures and algorithms that use atomic (without having to plaster if constexpr into their implementation, although we'd still have to plaster constexpr).

— Strongly Favor

Given that we have an implementation experience, I don't see a reason to hold this paper.

— Weakly Favor

This does not seem to be motivated.

— Weakly Against

§ 3.8. Poll 8: Send "[P3323R0] cv-qualified types in atomic and atomic_ref" to Library Working Group for C++26.

A much-awaited good resolution to library defects and implementation divergence.

— Strongly Favor

atomic_ref exists for use as a proxy reference type in an mdspan accessor. That works already, but this proposal makes it work even better. For example, it makes value_type consistent with its definition in mdspan for const-qualified element_type.

— Strongly Favor

Potential merge conflict between P3323R0/P3309R2 with respect to making sure that atomic_ref atomic operations are not considered constant expressions. Just needs to be brought to attention of LWG during wording review.

— Strongly Favor

atomic::notify_* operations for are non-const, while this paper does not add corresponding constraints to atomic_ref::notify operations. There is inconsistency as volatile atomic can always be constructed even if operations are not lock-free, and only invoking operations on it is ill-formed. This in particular allows is_lock_free to be checked on such an object. In contrast, atomic_ref template itself is ill-formed.

— Weakly Against

§ 3.9. Poll 9: Send "[P3325R3] A Utility for Creating Execution Environments" to Library Working Group for C++26.

Discussion in St. Louis made it clear why this utility needs to exist, and why it should be made available for users instead of being exposition-only or an implementation detail.

— Strongly Favor

We do need this functionality. Not often, but when we need it, it does need to be available in the standard.

— Strongly Favor

UX improvement for S/R.

— Weakly Favor

Given that we're going to have execution environments, this seems like a reasonable quality of life improvement to make it easier to manipulate them. However, execution environments seem to be resulting in a lot of design headaches currently being mulled over among S&R implementers and users.

— Neutral

§ 3.10. Poll 10: Send "[P0472R2] Put `std::monostate` in `<variant>` to Library Working Group for C++26.

`std::monostate` is a fabulously useful class that shouldn't require bringing in all of `<variant>` to use.

— Strongly Favor

Useful for user-defined variants, that want to also handle `monostate`, without bringing `std::variant`.

— Strongly Favor

Let's just do that if people see use cases.

— Weakly Favor

After 8 years, it doesn't seem likely that developers will start using `monostate` for things other than `variant`. It is hard to believe that the header it was in kept them away.

— Neutral

doesn't stay cheap by taking in all the random one-off classes.

— Neutral

§ 3.11. Poll 11: Send "[P3355R1] Fix `submdspan` for C++26" to Library Working Group for C++26.

This fixes a key use case of `mdspan`: iterating over compile-time-sized chunks of a run-time-sized array.

— Strongly Favor

The paper is very well-written and explains the rationale and technical bits with very high quality. It's another good example of well-done homework.

— Strongly Favor

§ 3.12. Poll 12: Send "[P3369R0] constexpr for uninitialized_default_construct" to Library Working Group for C++26.

With placement new being available at compile time, there is a tradeoff in making these algorithms available at compile-time.

— Strongly Favor

It doesn't touch parallel algorithms (which is good), so no reason to hold it.

— Weakly Favor

§ 3.13. Poll 13: Approve "[P3399R0] Adjusting Electronic Polls to a Hybrid Workflow" and apply the change to the process of Library Evolution Working Group.

People with design concerns who miss an LEWG live poll already have ways to be heard, both before and after the LEWG meeting where the live poll takes place. Electronic polls after the fact don't increase their ability to communicate their concerns.

— Strongly Favor

Electronic polling is helpful to gain additional feedback, and synchronize our async discussion. For face to face meetings it is less useful and would only cause unnecessary delays in a already slow process.

— Strongly Favor

It doesn't make much sense to vote twice for the same paper if the voting audience is also the same. I hope that everyone who can't participate to a given in-person vote but has opinion on a paper can express such opinions via email or similar, and that those concerns are addressed before a vote is then expressed.

— Weakly Favor

I see how this allows us to move faster during the last meetings of the standard, however, it requires active participation in the week-long meeting, which may not be feasible. The pools in contrast, can be handled asynchronously.

— Neutral

I like the opportunity to participate in electronic polls. I like how they give the people who weren't in the sessions the opportunity to cast their votes. I would hesitate to remove them from any of our processes, because I don't find the rationale for doing that convincing.

— Weakly Against

(...) The data in the paper indicates that this will disenfranchise a substantial portion of the people that were previously voting in electronic polls. (...) In person discussions do a poor job of representing past reflector discussions. (...) I am not receptive to the arguments that "the discussion is the valuable part of the process". While I agree that the discussion is more valuable than the voting, the voting still has substantial value. (...) I am not receptive to the arguments that "EWG stopped polling a while back". While those statements are true, that doesn't mean that those processes are better. We don't get better processes by always doing things the same way, or blindly following other's processes.

— Weakly Against

§ References

§ Informative References

[P3467R0]

Inbal Levi, Fabio Fracassi, Ben Craig, Nevin Liber, Billy Baker, Corentin Jabot. *2024-10 Library Evolution Polls*. 16 October 2024. URL: <https://wg21.link/p3467r0>